Mini Project 7 — Membangun Pipeline Pra-pemrosesan Baku

Mini Project 7 melanjutkan langsung Mini Project 6. Tidak ada perekaman ulang dan tidak ada perubahan sampling rate. Mahasiswa membaca audio filter final, melakukan DC removal, normalisasi, framing, VAD adaptif, spectral subtraction sederhana, trimming, membuat frame Hamming dengan pre-emphasis, menyimpan konfigurasi serta metrik, dan memperbarui status menjadi ready_for_mp08.

Tabel 7.8. Input dan output Mini Project 7.

Jenis	Path	Peran
Status input	project_status.json = ready_for_mp07	Validasi serial
Audio input	data/processed/mp06_design/nyala_001_filter_final.wav	Input resmi preprocessing
Filter config	config/filter_final_config.json	Menjaga provenance pipeline
Output config	config/preprocessing_config.json	Konfigurasi tunggal MP8
Output audio	data/processed/mp07_preprocessing/nyala_001_preprocessed.wav	Audit dan input alternatif
Output frame	features/intermediate/mp07_preprocessed_frames.npz	Input utama ekstraksi fitur
Metrik	results/metrics/mp07_preprocessing_metrics.json	Validasi proses
Status output	project_status.json = ready_for_mp08	Handoff serial

Langkah 1 — Validasi artefak MP6

Buka terminal pada folder audio_ai_embedded_project dan pastikan status proyek ready_for_mp07.

```
python -c "import json; s=json.load(open('project_status.json')); print(s['status']); assert s['status']=='ready_for_mp07'"
```

Langkah 2 — Buat skrip Mini Project 7

Buat file mini_project_07.py pada root proyek, lalu salin kode yang tersedia di bawah.

Langkah 3 — Jalankan preprocessing

Jalankan python mini_project_07.py. Skrip akan membuat konfigurasi, audio hasil, frame NPZ, metrik, dan dua gambar.

Langkah 4 — Periksa VAD dan waveform

Buka mp07_vad.png dan mp07_preprocessing_overview.png. Pastikan bagian aktif kata tidak terpotong dan output tidak clipping.

Langkah 5 — Verifikasi artefak serial

Periksa preprocessing_config.json dan project_status.json. Status harus ready_for_mp08.

Langkah 6 — Catat hasil singkat

Catat active_ratio, jumlah frame, durasi input/output, RMS sebelum/sesudah, dan perubahan noise-floor proxy pada lembar praktikum.

Kode Python Mini Project 7

```
from pathlib import Path
import json
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import stft, istft, welch

ROOT = Path('.')
STATUS = ROOT / 'project_status.json'

with open(STATUS, 'r', encoding='utf-8') as f:
    status = json.load(f)
assert status['status'] == 'ready_for_mp07', status['status']

input_wav = ROOT / status['mp07_input_audio']
filter_cfg = ROOT / status['filter_final_config']
assert input_wav.exists(), input_wav
assert filter_cfg.exists(), filter_cfg

fs, x = wavfile.read(input_wav)
if x.ndim > 1:
    x = x.mean(axis=1)
if np.issubdtype(x.dtype, np.integer):
    scale = max(abs(np.iinfo(x.dtype).min), np.iinfo(x.dtype).max)
    x = x.astype(np.float64) / scale
else:
    x = x.astype(np.float64)

# 1) DC removal + peak normalization
x_dc = x - np.mean(x)
target_peak = 0.95
x_norm = target_peak * x_dc / (np.max(np.abs(x_dc)) + 1e-12)

# 2) Framing parameters
frame_ms = 25.0
hop_ms = 10.0
frame_len = int(round(fs * frame_ms / 1000.0))
hop_len = int(round(fs * hop_ms / 1000.0))
window = np.hamming(frame_len)

starts = np.arange(0, max(1, len(x_norm) - frame_len + 1), hop_len)
if len(starts) == 0:
    starts = np.array([0])
frames = []
for s in starts:
    fr = x_norm[s:s+frame_len]
    if len(fr) < frame_len:
        fr = np.pad(fr, (0, frame_len-len(fr)))
    frames.append(fr)
frames = np.asarray(frames)
rms = np.sqrt(np.mean(frames**2, axis=1) + 1e-12)

# 3) Adaptive VAD
noise_rms = float(np.percentile(rms, 10))
high_rms = float(np.percentile(rms, 90))
vad_factor = 0.25
vad_threshold = noise_rms + vad_factor * max(0.0, high_rms - noise_rms)
active = rms >= vad_threshold
# Fallback konservatif: jangan biarkan VAD memotong hampir seluruh kata.
if np.mean(active) < 0.15:
    vad_threshold = float(np.percentile(rms, 50))
    active = rms >= vad_threshold
if not np.any(active):
    active[:] = True
raw_active = active.copy()
noise_reliable = bool((np.mean(~raw_active) >= 0.10) and (high_rms >= 1.8 * max(noise_rms, 1e-12)))

# Tambahkan margin 2 frame sebelum/sesudah area aktif
idx = np.flatnonzero(active)
first = max(0, int(idx[0]) - 2)
last = min(len(frames)-1, int(idx[-1]) + 2)
active[first:last+1] = True

# 4) Spectral subtraction sederhana
nperseg = min(512, frame_len)
noverlap = int(0.5 * nperseg)
f, tt, Z = stft(x_norm, fs=fs, window='hann', nperseg=nperseg,
                noverlap=noverlap, boundary='zeros')
```

```
mag = np.abs(Z)
phase = np.exp(1j * np.angle(Z))

# Estimasi noise dari kolom STFT dengan energi terendah
col_energy = np.sqrt(np.mean(mag**2, axis=0) + 1e-12)
q = max(1, int(round(0.20 * len(col_energy))))
noise_cols = np.argsort(col_energy)[:q]
noise_mag = np.median(mag[:, noise_cols], axis=1, keepdims=True)

beta = 0.6 if noise_reliable else 0.0
spectral_floor = 0.25
clean_mag = np.maximum(mag - beta * noise_mag,
                       spectral_floor * noise_mag) if beta > 0 else mag.copy()
_, y = istft(clean_mag * phase, fs=fs, window='hann', nperseg=nperseg,
             noverlap=noverlap, input_onesided=True, boundary=True)
y = y[:len(x_norm)]

# 5) Trim memakai batas VAD
sample_start = int(starts[first])
sample_end = int(min(len(y), starts[last] + frame_len))
y_trim = y[sample_start:sample_end]
y_trim = target_peak * y_trim / (np.max(np.abs(y_trim)) + 1e-12)

# 6) Frame output + pre-emphasis untuk Mini Project 8
starts2 = np.arange(0, max(1, len(y_trim)-frame_len+1), hop_len)
if len(starts2) == 0:
   starts2 = np.array([0])
frames2=[]
for s in starts2:
   fr=y_trim[s:s+frame_len]
   if len(fr)<frame_len:
      fr=np.pad(fr,(0,frame_len-len(fr)))
   frames2.append(fr)
frames2=np.asarray(frames2)
alpha=0.97
pre = frames2.copy()
pre[:,1:] = frames2[:,1:] - alpha*frames2[:,:-1]
pre[:,0] = frames2[:,0]
windowed = pre * window[None,:]

# 7) Metrics
rms_in=float(np.sqrt(np.mean(x_norm**2)+1e-12))
rms_out=float(np.sqrt(np.mean(y_trim**2)+1e-12))
active_ratio=float(np.mean(active))
clipping=int(np.sum(np.abs(y_trim)>=0.999))

# Noise-floor proxy: median PSD 3–7 kHz
fi, pi = welch(x_norm, fs=fs, nperseg=min(1024,len(x_norm)))
fo, po = welch(y_trim, fs=fs, nperseg=min(1024,len(y_trim)))
def hf_floor(freq, psd):
   mask=(freq>=3000)&(freq<=7000)
   if not np.any(mask): return float(np.median(psd))
   return float(np.median(psd[mask]))
noise_in=hf_floor(fi,pi)
noise_out=hf_floor(fo,po)
noise_reduction_db=float(10*np.log10((noise_in+1e-15)/(noise_out+1e-15)))

cfg={
   'sampling_rate_hz': int(fs),
   'dc_removal': True,
   'normalization': {'type':'peak','target_peak':target_peak},
   'frame_ms': frame_ms,
   'hop_ms': hop_ms,
   'frame_length_samples': frame_len,
   'hop_length_samples': hop_len,
   'window': 'hamming',
   'pre_emphasis_alpha': alpha,
   'vad': {'method':'frame_rms_adaptive','relative_factor':vad_factor,
         'threshold':float(vad_threshold),'margin_frames':2},
   'denoise': {'method':'spectral_subtraction','beta':beta,
            'spectral_floor':spectral_floor,'noise_fraction':0.20,
            'noise_estimate_reliable':noise_reliable},
   'source_filter_config': str(filter_cfg.relative_to(ROOT))
}

out_audio=ROOT/'data/processed/mp07_preprocessing'; out_audio.mkdir(parents=True,exist_ok=True)
feat_dir=ROOT/'features/intermediate'; feat_dir.mkdir(parents=True,exist_ok=True)
metric_dir=ROOT/'results/metrics'; metric_dir.mkdir(parents=True,exist_ok=True)
fig_dir=ROOT/'results/figures'; fig_dir.mkdir(parents=True,exist_ok=True)
config_dir=ROOT/'config'; config_dir.mkdir(parents=True,exist_ok=True)

wav_out=out_audio/'nyala_001_preprocessed.wav'
wavfile.write(wav_out, fs, np.int16(np.clip(y_trim,-1,1)*32767))
np.savez(feat_dir/'mp07_preprocessed_frames.npz',
       frames=windowed.astype(np.float32), fs=np.int32(fs),
       frame_length=np.int32(frame_len), hop_length=np.int32(hop_len),
       pre_emphasis_alpha=np.float32(alpha), window='hamming')
with open(config_dir/'preprocessing_config.json','w',encoding='utf-8') as f:
   json.dump(cfg,f,indent=2)
```

```
metrics={
  'input_duration_s': float(len(x_norm)/fs),
  'output_duration_s': float(len(y_trim)/fs),
  'input_rms': rms_in,
  'output_rms': rms_out,
  'vad_threshold': float(vad_threshold),
  'active_ratio': active_ratio,
  'n_input_frames': int(len(frames)),
  'n_output_frames': int(len(frames2)),
  'noise_floor_change_db_proxy': noise_reduction_db,
  'output_clipping_samples': clipping
}
with open(metric_dir/'mp07_preprocessing_metrics.json','w',encoding='utf-8') as f:
  json.dump(metrics,f,indent=2)

# Figures
fig,ax=plt.subplots(figsize=(10,4.5))
ax.plot(rms,label='RMS frame')
ax.axhline(vad_threshold,ls='--',label='VAD threshold')
ax.fill_between(np.arange(len(rms)),0,rms,where=active,alpha=.2,label='active')
ax.set_xlabel('Frame'); ax.set_ylabel('RMS'); ax.grid(alpha=.2); ax.legend()
fig.tight_layout(); fig.savefig(fig_dir/'mp07_vad.png',dpi=180); plt.close(fig)

fig,axs=plt.subplots(2,1,figsize=(10,6.2))
axs[0].plot(np.arange(len(x_norm))/fs,x_norm,label='Input MP6',lw=.8)
axs[0].plot(np.arange(sample_start,sample_end)/fs,y_trim,label='Output MP7',lw=.8)
axs[0].set_ylabel('Amplitude'); axs[0].grid(alpha=.2); axs[0].legend()
axs[1].semilogy(fi,pi+1e-15,label='Input PSD')
axs[1].semilogy(fo,po+1e-15,label='Output PSD')
axs[1].set_xlim(0,fs/2); axs[1].set_xlabel('Frequency (Hz)'); axs[1].set_ylabel('PSD')
axs[1].grid(alpha=.2); axs[1].legend()
fig.tight_layout(); fig.savefig(fig_dir/'mp07_preprocessing_overview.png',dpi=180); plt.close(fig)

# 8) Update serial status
status.update({
  'status':'ready_for_mp08',
  'last_completed_mini_project':7,
  'preprocessing_config':'config/preprocessing_config.json',
  'preprocessed_audio':'data/processed/mp07_preprocessing/nyala_001_preprocessed.wav',
  'preprocessed_frames':'features/intermediate/mp07_preprocessed_frames.npz',
  'preprocessing_metrics':'results/metrics/mp07_preprocessing_metrics.json',
  'mp08_input_audio':'data/processed/mp07_preprocessing/nyala_001_preprocessed.wav',
  'mp08_input_frames':'features/intermediate/mp07_preprocessed_frames.npz'
})
with open(STATUS,'w',encoding='utf-8') as f:
  json.dump(status,f,indent=2)

print('Input duration :', round(metrics['input_duration_s'],3),'s')
print('Output duration:', round(metrics['output_duration_s'],3),'s')
print('Active ratio  :', round(active_ratio,3))
print('Output frames  :', len(frames2))
print('Noise-floor change proxy:', round(noise_reduction_db,2),'dB')
print('Denoise applied:', beta > 0)
print('Clipping samples:', clipping)
print('Status: ready_for_mp08')
```

Kriteria Lulus Mini Project 7

Tabel 7.9. Checklist kelulusan dan handoff ke Mini Project 8.

Artefak / pemeriksaan	Kriteria lulus	Dipakai MP8
project_status.json	status = ready_for_mp08	Ya
preprocessing_config.json	fs=16000; frame=25 ms; hop=10 ms; window=Hamming	Ya
nyala_001_preprocessed.wav	Dapat diputar; tidak clipping; durasi > 0	Ya
mp07_preprocessed_frames.npz	Array frame 2D; tanpa NaN/Inf	Ya, input utama
mp07_preprocessing_metrics.json	active_ratio dan jumlah frame tercatat	Audit
mp07_vad.png	Wilayah aktif kata tidak terpotong	Audit
mp07_preprocessing_overview.png	Waveform dan PSD dapat dibandingkan	Audit

Handoff wajib ke Mini Project 8

Mini Project 8 harus membaca mp08_input_frames dan preprocessing_config dari project_status.json. Jangan melakukan framing, windowing, pre-emphasis, atau VAD ulang dengan parameter baru. MP8 hanya mengekstraksi dan membandingkan fitur dari representasi yang telah dibakukan di Bab 7.

Referensi Bab

1. Oppenheim, A. V., & Schafer, R. W. Discrete-Time Signal Processing. Pearson.

2. Rabiner, L. R., & Schafer, R. W. Theory and Applications of Digital Speech Processing. Pearson.

3. Loizou, P. C. Speech Enhancement: Theory and Practice. CRC Press.

4. Boll, S. F. Suppression of acoustic noise in speech using spectral subtraction. IEEE Transactions on Acoustics, Speech, and Signal Processing.

5. Davis, S., & Mermelstein, P. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences.

6. Dokumentasi SciPy Signal Processing untuk STFT, ISTFT, Welch PSD, serta operasi numerik yang digunakan pada praktikum.